

CS780 Capstone Project: OBELIX: the Warehouse Robot

Pankaj Bhanu
242110607
{bpankaj24}@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

The project presented us with a POMDP warehouse robot which had to discover and push a box out of the arena. Our experiments suggested that SAC and PPO were the best algorithms to achieve the objective with PPO being easier to implement and yielded better results.

1 Competition Result

Codabench Username: p_242110607
Final leaderboard rank (Test Phase): 72 (-8906.6)
Leaderboard rank – Level 1 (static): 92 (-17760)
Leaderboard rank – Level 2 (blinking): 57 (-1981.2)
Leaderboard rank – Level 3 (moving + blinking): 62 (-1515.4)

Valid submissions made (Development(Level 1 , Level 2, Level 3) and Test):

Table 1: Submission Scores across Phases

Phase	Scores
1	[-22221, -120647, -80965, -394476, -17760.60, -220323, -52158, -17128.10]
2	[-1981.20, -72708.70]
3	[-85749.92, -80400.16, -1515.36]
4	[-8906.59, -16507.27, -1828.51]

2 Introduction and Problem Description

- Q. Briefly introduce the motivation.
- A. Robots are the next phase of industrial revolution. This project aims to prove that a robot equipped with only basic IR sensors (which are noisy and low-resolution) can perform sophisticated navigation in a warehouse.
- Q. Describe the core task: find $\rightarrow$ attach $\rightarrow$ push $\rightarrow$ unwedge grey box to boundary
- A. The core task is to make the robot move in a rectangular confinement and make it discover the box using its sensors. The next task is to wedge the box and push it out of the arena.
- Q. Explain the environment.

- A. The environment consists of a rectangular arena along with partition walls in the middle. Hitting areas that are on the boundary or the partition wall yields -200 reward. There is also a box present in the arena. Discovering it through sensors yields small (< 10) positive rewards. Wedging the box results in a +100 reward and pushing it out of the arena yields +2000 reward. The survival reward is -1 per time step (max 2000 steps).
- Q. Describe the three difficulty levels (static, blinking, moving+blinking) and optional wall obstacle.
- A. The first difficulty level "static" refers to when the box is stationary at a random position in the arena throughout the episode. The second difficulty level "blinking" refers to when the box is stationary but blinking at a random position in the arena. Blinking makes the box invisible to the robot sensors. The third difficulty is when within one episode the box moves throughout the arena and blinks as well. There are half-partition walls in the middle of the arena which block the robot's path and give -200 reward on collision.
- Q. Mention that this is a POMDP setting and why it is challenging.
- A. The Obelix robot operates in a Partially Observable Markov Decision Process (POMDP) environment because the information available to the agent at any single moment is insufficient to determine the robot's true state in the arena. In a standard MDP, the current observation provides everything needed to make an optimal decision. In a POMDP, the "Markov Property" is broken because the robot's history matters.
- Q. State your high-level goal: maximize mean cumulative reward across difficulties and robustness to walls.
- A. The high level goal is to navigate in the arena without colliding onto walls or boundaries in search of the box. When the box is discovered, the next goal is to push it out of the arena. The scores are cumulative of 10 episodes with two scenarios: with wall and without wall.

3 Approach and Model Evolution

3.1 Baseline

The baseline model was obtained through the example DQN network shared along with the obelix environment files. The architecture consisted of 18 dimensional input layer, two hidden layers of 64 dimensions each. The output layer had 5 dimensions.

3.2 Phase 1

In order to solve this POMDP environment, the initial strategy was to use a Double Deep Q network. The architecture was same as baseline model above. Without any reward shaping or hand-crafted features, the robot settled on rotating and idling in its spawn position after training for 3000 episodes.

The next strategy was to use Dueling DQN with a Prioritized Experience Replay (PER) so that the robot could learn from the rare positive rewards that it got while wedging or throwing the box as well as the negative rewards of collision. The architecture was modified to: 18 dimensional input layer, two hidden layers of 256 and 128 dimensions respectively and 5 dimensional output layer. An epsilon-greedy exploration strategy was also added. The bot again settled for idle circling after training for 3000 episodes.

After two unsuccessful attempts to converge to the correct strategy, the next idea was to augment the 18 raw sensor data with hand-crafted features like consecutive forward moves, consecutive turns, etc were added to make it a 24 dimensional input layer, 512 and 256 dimensional hidden layers and 5 dimensional output layer. Also now action-advantage function was added to calculate loss during forward pass. During training the bot showed some movement, but during evaluation it still resorted to circling.

3.3 Phase 2

Now, the result of previous two attempts revealed that the POMDP had a sparse reward problem. The rewards obtained by the bot were so sparse during a 2000 step episode that the bot was unable to assign proper credit to the correct action. Also it wanted to avoid the negative rewards that it accumulated while trying to explore which resulted in idle circling. So the next strategy was to further add hand-crafted features which increased their number to 38. Some of them are:

- Sum of front near/far sensor bits
- Sum of left near/far sensor bits
- Sum of right near/far sensor bits
- Consecutive turns
- Consecutive forward moves
- Rotational fatigue
- Lissajous curve parameters with short cycle 80 and long cycle 400

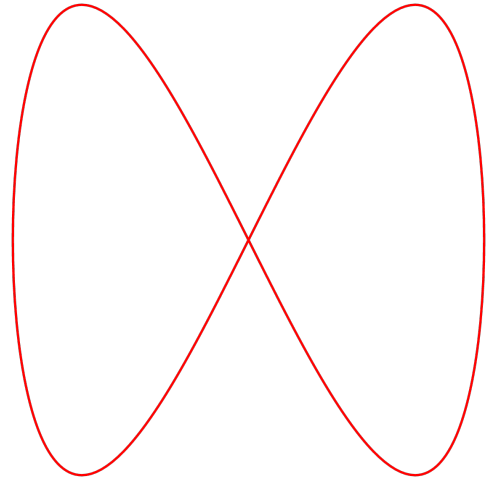


Figure 1: Agent Trajectory Pattern

Along with this, reward shaping was added so that credit assignment to the hand-crafted features could be improved. The training trajectory of the bot improved drastically. Now it roamed around the arena avoiding getting stuck on walls and following the Lissajous pattern in the long horizon. The evaluation was still not-satisfactory as it was missing epsilon-greedy exploration and the reshaped rewards.

The next idea was to use these hand-crafted features and reward shaping in a Proximal Policy Optimization (PPO) algorithm. PPO is an on-policy strategy which rewards exploration by adding an entropy term to the loss function. The architecture was changed to 40 input dimensions, 512 and 256 hidden dimensions and 5 output dimensions. This improved the training a lot, but if you allowed the bot to train for 3000 episodes, the converged weights had again started to encourage idle circling. The weights obtained midway e.g. after 250 episodes still favoured exploration.

3.4 Phase 3

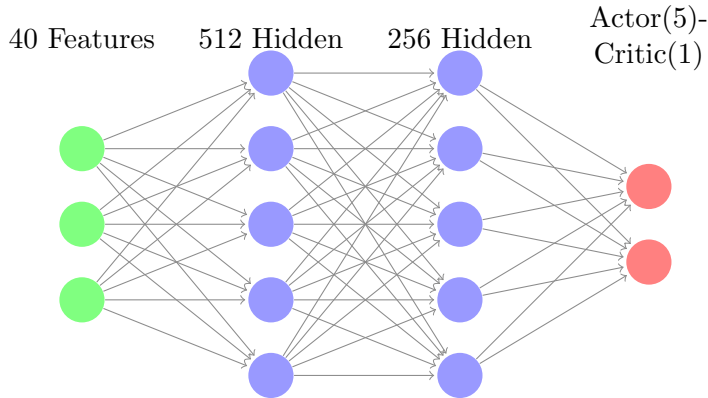
The next attempt was along the similar line of using Actor-Critic framework, i.e. Soft-Actor Critic. This is again an off-policy algorithm like DDQN. So a Prioritized Experience Replay with SumTree based priority extraction of buffer was added. The architecture was kept similar to PPO along with the reward shaping and 40 (18 raw and 22 augmented) features. We included Dead Reckoning (DR) to check if the bot was stuck in the arena. DR is a technique to calculate relative position from a starting unknown position using one's velocity and angle of motion. Due to the presence of walls and boundary DR can only be done for few steps as it is prone to cumulative error or drift. Further action-biasing was added to the neural network directly to force it out of getting stuck in no-sensor void zones, corners and idle spinning. The bot really tried and continued to explore the environment this time, even in later episodes. There was some dizziness (unable to find the right direction to get out of stuck zone) around the corners. This resulted in a high collision-count and much lower negative cumulative score for the training and evaluation episodes. The evaluation however revealed that the bot was now not idling and exploring the arena getting stuck once in a while.

3.5 Phase 4

In phase 4, we again tried to stick to the basics and reverted back to PPO. We removed the reward shaping but kept scenario based action biasing. We tried to add a decay to the action biasing, but once decay phase ended the bot still started getting stuck in no-sensor zones and corners. The NN architecture follows the same as SAC with its 40 features. Generalized Action Advantage function was used. We were surprised to see that, even in the absence of reward shaping the bot continued to explore the environment even in the evaluation episodes. As expected, there were collisions with walls and boundaries but the collision clearance was much smoother, i.e., the bot was decisive in which direction to turn in no-sensor zones and corners. This phase resulted in a much better score for an explorative bot and was the best excluding the idle spinning and reward farming.

3.6 Detailed description of final / best model

- Algorithm family: Proximal Policy Optimization (PPO) was used along with Generalized Advantage Estimation to balance the trade-off between bias and variance when estimating how much **better** an action is compared to the average behavior.
- PPO Network Architecture is described below:



- State representation: A hand-crafted feature-set of size 40 (18 raw, 22 crafted) was used as input to the Neural Network. Some of the features have been described previously in Phase 2.
- Loss Function: The Actor loss function is the minimum of the mean of the two surrogate losses. The Critic loss is the mse loss of the predicted value and the actual return. There is an entropy bonus with a entropy factor of 0.08 also in the total loss function. This boosts the chances of converging to a policy which explores all possible actions.
- Minor (almost nil) reward shaping is used in the final model.

4 Experiments and Implementation Details (Final Model)

- Training setup: Training consisted of 4000 episodes, of max 2000 steps. Hardware consisted of Intel Core i9-14900. The model training time was 6-8 hours.
- Adam Optimizer with a learning rate of 1e-4 was used. Discount factor (γ) was 0.99. Generalized Advantage Estimation was used with $\lambda = 0.95$.
- Hyperparameter table (one per major model variant) is summarized in 2.
- The model was switched to evaluation mode and the mean rewards of ten episodes (10 seeds) was calculated to check for overfitting.

Table 2: Hyperparameters for OBELIX Training Experiments

Algorithm	Hyperparameter	Value
DDDQN (Dueling Double DQN)	Learning Rate	1×10^{-4}
	Batch Size	1024
	Gamma (γ)	0.99
	Tau (τ - Soft Update)	0.005
	Epsilon (Start/End/Decay)	1.0 / 0.01 / 400 steps
	Target Net Update	Every 20 steps (Soft)
	Replay Buffer	500,000 (Prioritized)
	Architecture	Dueling (512, 256, 128)
SAC (Soft Actor-Critic)	Actor Learning Rate	8×10^{-5}
	Critic Learning Rate	3×10^{-4}
	Batch Size	256
	Gamma (γ)	0.99
	Tau (τ)	0.005
	Alpha (α)	0.2
	Replay Buffer Size	100,000
	Hidden Layers	[512, 256]
PPO (GAE) (Proximal Policy Optimization)	Actor Learning Rate	1×10^{-4}
	Gamma (γ)	0.99
	GAE Lambda (λ)	0.95
	PPO Clip (ϵ)	0.2
	PPO Epochs	4
	Entropy Coefficient (c_2)	0.08
	Hidden Layers	[512, 256]

5 Results

The performance of different models/variants on development phases 1, 2, 3 and 4 are summarized in 3. The final model learning curves are given in section 6. The reason why PPO performed best are given below:

- **Robustness via Clipped Objectives:** PPO’s clipped surrogate objective prevented the agent from making drastically large policy updates. This was critical in preventing the ”idle circling” behavior observed when the agent encountered high-penalty regions during exploration.
- **Credit Assignment with GAE:** GAE allowed the agent to efficiently propagate rewards back through the 2000-step episodes. This smoothed the sparse reward signal, allowing the bot to associate early sensor patterns with eventual success.
- **Sample Efficiency in Discrete Action Spaces:** PPO proved highly effective for the five-action discrete set. The ability to perform multiple(4) epochs of updates on a single collection of rollout data allowed the model to extract maximum learning value from every interaction.
- **Generalization Across Difficulty Levels:** While Difficulty 0 and 2 were simple, Difficulty 3 introduced obstacle movement. PPO’s stability ensured that the agent could adapt its ”forward-moving” bias while clearing collisions without collapsing the entire policy.

Table 3: Agent Performance Benchmarks

Agent Name	Difficulty	Walls	Mean Score	Std Dev	Runs	Box Speed	Arena
D3QN (Phase 1)	0	True	-1979.4	6.3	10	10	500
D3QN (Phase 1)	0	False	-1997.2	4.8	10	10	500
D3QN (Phase 1)	2	True	-1979.4	6.3	10	10	500
D3QN (Phase 1)	2	False	-1997.2	4.8	10	10	500
D3QN (Phase 1)	3	True	-747.7	1715.7	10	10	500
D3QN (Phase 1)	3	False	-1483.2	1189.8	10	10	500
PPO (Phase 2)	0	True	-1979.4	6.3	10	10	500
PPO (Phase 2)	0	False	-1997.2	4.8	10	10	500
PPO (Phase 2)	2	True	-1979.4	6.3	10	10	500
PPO (Phase 2)	2	False	-1997.2	4.8	10	10	500
PPO (Phase 2)	3	True	-747.9	1715.4	10	10	500
PPO (Phase 2)	3	False	-1483.4	1189.8	10	10	500
SAC (Phase 3)	0	True	-134123.7	29330.5	10	10	500
SAC (Phase 3)	0	False	-74977.8	49189.7	10	10	500
SAC (Phase 3)	2	True	-136102.9	43494.4	10	10	500
SAC (Phase 3)	2	False	-67477.1	47290.8	10	10	500
SAC (Phase 3)	3	True	-41720.2	27082.5	10	10	500
SAC (Phase 3)	3	False	-21586.9	19330.3	10	10	500
PPO (Phase 4)	0	True	-11660.4	8266.4	10	10	500
PPO (Phase 4)	0	False	-2848.5	3656.4	10	10	500
PPO (Phase 4)	2	True	-9833.4	13260.1	10	10	500
PPO (Phase 4)	2	False	-4941.8	5330.8	10	10	500
PPO (Phase 4)	3	True	-12216.7	11756.7	10	10	500
PPO (Phase 4)	3	False	-2758.0	5440.6	10	10	500

6 Training Progress Analysis

The following figures illustrate the convergence behavior of the PPO agent during the training phase. These metrics were recorded across 5,000 episodes, focusing on policy stability, value estimation accuracy, and exploration dynamics.

6.1 Metric Interpretation

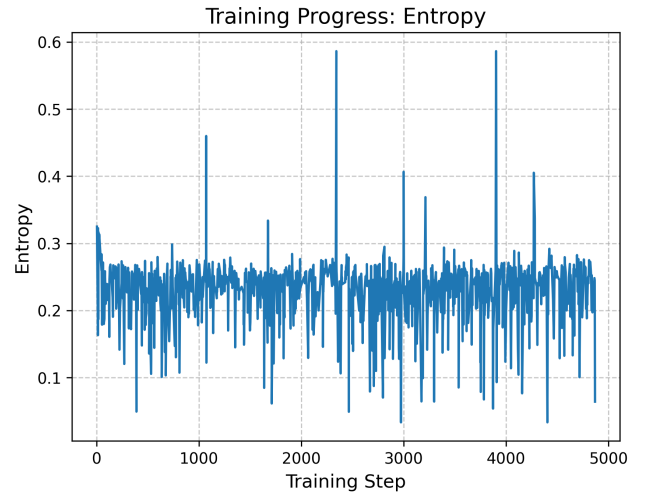
As seen in Figure 2, the agent demonstrates consistent policy refinement. The steady running average in Figure 2b confirms that the agent is successfully transitioning from exploratory actions to a more consistent reward policy.

7 Error Analysis and Discussion

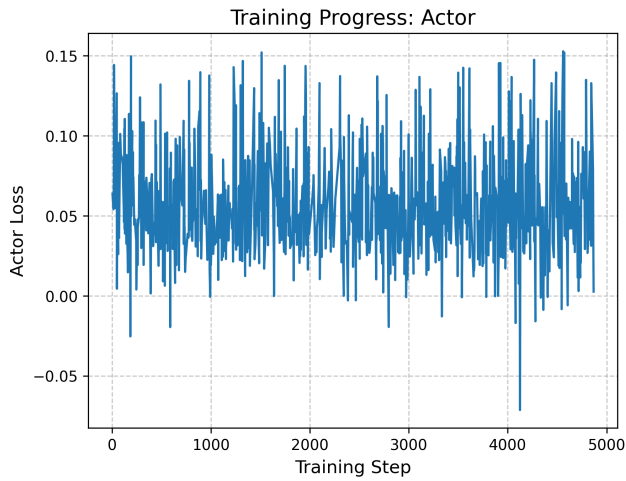
- Common failure modes per difficulty level
 - Level 1: The bot fails even in presence of static box because, the sensor signature of box and the wall is same. Closing in onto the wall, thinking its a box results in heavy penalty. This prevents the bot from exploring which is detrimental especially when box is on the other side of the wall.



(a) Cumulative Episode Reward



(b) Policy Entropy



(c) Actor Loss



(d) Critic Loss (MSE)

Figure 2: PPO Training Metrics for the OBELIX Robot. The reward curve shows the primary optimization objective, while Entropy, Actor Loss, and Critic Loss provide information regarding the internal optimization state.

- Level 2: The blinking box confuses the bot as it gives false positives. The bot on treading through the box can't find it and moves away.
- Level 3: Level turned out to be sometimes easier than Level 2. If you kept your exploration region confined to a small polygon, the box would sometimes come to you and get wedged. Now it could easily be pushed out of the arena. Following the box's trajectory, however, was very difficult since its speed was comparable to the bot's speed.
- With walls: Having walls was the most challenging environment in all stages since the wall length exceeds almost $\frac{3}{4}th$ of the height of the arena. This prevented the bot from exploring the other side of the wall. Also the sensor signature of box and wall were identical resulting in heavy penalty collisions.
- The most prominent limitation of the bot is it can't distinguish wall from box. Just sonar signals are not enough, some form of vision information from camera is required. With more time, we could use CNN to feed the arena pixels and then used DDQN to drive the bot to the box like in Atari games.
- The insightful thing about this project was that both off-policy(SAC) and on-policy(PPO) algorithms could converge to a similar policy given sufficient training and similar reward structure.

8 Conclusion

- The main achievement was to be able to implement prominent RL algorithms SAC or PPO even being a novice in RL area. PPO turned out to be a surprisingly simple but powerful algorithm. It could adapt to the environment with and without reward shaping.
- The most critical takeaway is that in partially observable robotic tasks, the agent cannot rely on a single snapshot of sensor data to make an optimal decision; it must instead maintain a belief state or temporal memory to infer the hidden variables of the environment.
- POMDP methods benefit from Recurrent Neural Networks (DRQN). Replacing current MLP layers with LSTM or GRU cells would allow the agent to maintain a latent memory state. This is more robust than manual counters as the network learns what to remember.

9 Deliverables

- The link for demo of the final phase PPO is available at <https://youtu.be/K3mFN4MGyPA>
- The link for the github repo for training routines are available at <https://github.com/pankajbhanu/drl-obelix>

10 LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report? Gemini and Chatgpt were used to explore about Reinforcement Learning algorithms and concepts.
- In which part did you use the LLM, and how did it help you? LLMs were used to get a better understanding of the Loss functions used in algorithms like SAC or PPO.

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Understanding Loss Functions
Ideation / brainstorming	No	
Debugging small code snippets	Yes	Indexing errors, Bug fixes Plotting csv log files
Plotting / visualization code	Yes	
Grammar / writing style	No	
Report structure / section ideas	No	
Other (specify)		

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
If yes, please mention clearly (papers, blogs, GitHub repos, textbooks etc.)
Yes. We checked the original SAC[1] and PPO[2] papers just to get an idea on their applicability to the current POMDP. We used the book Grokking Deep Reinforcement Learning by Miguel Morales [3] and the github repo by sweetice[4].
- Apart from any LLM usage, what was your own contribution? Briefly describe your personal understanding, reasoning, implementation effort, analysis, and experimentation.
The entire ideation, implementation, network design and hyperparameter tuning was done without LLM.

References

- [1] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” 2017.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [3] mimoralea, “Grokking deep reinforcement learning,” 2022.
- [4] sweetice, “Deep-reinforcement-learning-with-pytorch,” 2019.